StatCore Meta-Learning: Cross-Fitted Model Combination and NumPy Reliability Metrics

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—This paper specifies the StatCore `MetaLearner`, a NumPy-based classifier that combines named probabilistic (or hard-label) classifiers using out-of-fold (OOF) predictions. We state the implemented behavior of five alternatives—`performance_aware`, `metrics_based`, `oof_stacking`, `statistical_meta`, and `calibrated_stack`—and contrast them with conventional stacked generalization and scikit-learn's `StackingClassifier`. We distinguish the numerical StatCore implementation from scikit-learn-compatible adapters, which use scikit-learn for estimator protocol and validation while delegating numerical classification to StatCore. We define the shared classification, calibration, regression, reconstruction, and clustering metrics, including the implementation's edge-case conventions. Equations and algorithm blocks are implementation-level descriptions, not claims of statistical optimality. OOF construction limits one form of training leakage but does not remove model-selection optimism or distribution shift.**

***Index Terms*—statistical learning, stacked generalization, cross-fitting, calibration, NumPy, model selection**

I. Introduction

Stacked generalization trains a second-stage predictor from predictions of first-stage learners [1], [2]. A stack trained on in-sample base predictions can exploit optimism unavailable at deployment. A standard remedy is to construct meta-training inputs from held-out folds. The StatCore implementation applies this pattern in a pure-NumPy core and supplies five selectable strategies. Here "meta-learning" denotes supervised combination and selection among models; it is not task-distribution learning or few-shot adaptation.

The implementation source of record is `statcore/meta/meta_learning.py`; common validation and deterministic folds are in `statcore/_common.py`; metric definitions are in `statcore/evaluation/metrics.py`. `statcore/SPEC.md` documents the intended model-family contract. The software also exposes sklearn-style wrappers in `statcore_estimators.py`. Only these adapters depend on sklearn's estimator classes and validation utilities; the StatCore numeric classifiers and meta-combination computations use NumPy.

II. Notation and common cross-fitting

Let $D = \{(x_i, y_i)\}_{i=1}^{n}$, sorted unique class vector $\mathcal{C} = (c_1, \ldots, c_C)$, M named base estimators, and $K \geq 2$ stratified folds. Fold IDs are made reproducibly using a NumPy generator initialized with `random_state`; within each class, shuffled indices are assigned cyclically over fold IDs. For fold k, base estimator m is deep-copied and fit on D_{-k}. Its predictions on D_k are aligned to the complete training class order. With probability output $P_{im} \in [0,1]^C$; if only `predict` exists, the implementation substitutes a one-hot vector for the predicted class. A class absent from a fold-training partition contributes a zero column. This fallback permits protocol compatibility but is not a calibrated probability estimate.

The OOF tensor is $O \in \mathbb{R}^{M \times n \times C}$, with rows $O_{m,i,:} = P_{im}$. Each training sample's base prediction is generated by a model whose fit excluded that sample. Then, independently, every original base estimator is deep-copied and refit on all n examples for inference. This means training meta-features originate from fold-specific models whereas test-time features originate from full-data models, a finite-sample train/deploy mismatch inherent in ordinary cross-fitted stacking. Inputs must be finite, numeric, two-dimensional, with matching one-dimensional labels; at least two classes are required. Fold construction does not guarantee every training fold retains

every class; code raises when a fold-training set has fewer than two classes.

Define the clipped logarithm $\log_\epsilon(p) = \log(\max(p, 10^{-12}))$ for entropy and the metric code's clipping to $[10^{-12}, 1]$ for log-loss. The per-row entropy is $H(P_i) = -\sum_c P_{ic} \log_\epsilon(P_{ic})$. A multiclass softmax is stabilized by subtracting the row maximum before exponentiation.

III. Five meta variants

A. Performance-aware weighting

For base m, the code computes OOF multiclass log-loss $L_m = -n^{-1} \sum_i \log(\max(O_{m,i,y_i}, 10^{-12}))$. With user parameter $\tau = \texttt{temperature} > 0$, it sets

$$w_m = \frac{\exp(-(L_m - \min_j L_j)/\tau)}{\sum_{r=1}^{M} \exp(-(L_r - \min_j L_j)/\tau)},$$
$$\hat{P}(y \mid x) = \sum_{m=1}^{M} w_m P_m(x).$$

The argmin-loss estimator name is retained as `best_estimator_`; the ensemble prediction uses all softmax weights, not a hard selection. Smaller τ concentrates weight on the best observed OOF loss. This score-dependent combination has no fitted second-stage classifier. It uses OOF labels to estimate weights and therefore its reported training OOF loss is not an independent assessment of the final weighted predictor.

B. Metrics-based learned combination

Let $\bar{P}_i = M^{-1} \sum_m O_{m,i,:}$, and $a_i = \arg\max_c \bar{P}_{ic}$ be consensus class. For each estimator, calculate OOF macro-F1, F_m, from hard predictions. The design matrix concatenates (i) each probability vector scaled by its estimator's single global OOF macro-F1, $F_m O_{m,i,:}$; (ii) per-model confidence, entropy, and top-two probability margin $q_{im} = (P_{(C),im}, H(P_{im}), P_{(C),im} - P_{(C-1),im})$, where sorted order is ascending; and (iii) row-wise fraction of models agreeing with consensus plus entropy of mean probabilities. For $C = 1$, margin is the top probability; fitted classification in practice needs at least two classes. A logistic regression with $\alpha = 10^{-2}$ and configured seed is trained on these features and y. Its learned combination is therefore not equivalent to static performance weighting. Since F_m is computed over all OOF rows, each row contributes to a global feature scale; this is not a second independent validation layer.

C. Ordinary OOF probability stacking

For this variant, $Z_i = [O_{1,i,:}^\top, \ldots, O_{M,i,:}^\top]^\top \in \mathbb{R}^{MC}$. A StatCore multinomial logistic model ($\alpha = 10^{-2}$) is fit to (Z_i, y_i). At inference the full-data base models construct the analogous concatenation and the logistic meta-classifier returns probabilities. This is the classic stacked-generalization formulation, with cross-fitted rather than in-sample meta-training predictions [1], [2]. It differs from the sklearn `StackingClassifier` only in implementation choices and underlying estimator protocol; the latter is a useful standard baseline, not an algorithmic novelty claim [3].

D. Statistical meta-ensemble

For each base probability vector, the features are elementwise clipped log-probabilities; thus $U_i = [\log(\max(O_{1,i,:}, 10^{-3}))^\top, \ldots]^\top$. This is a transform of OOF base outputs, not raw X. Three distinct learners are fitted to (U, y): shrinkage LDA, StatCore KMeans (`better_init`) with $k = \max(C, \min(2C, \lfloor n/5 \rfloor))$, and logistic regression ($\alpha = 10^{-2}$). Let r_{ic} be class index of y_i, and cluster assignment g_i. Cluster posterior table uses additive-one counts

$$Q_{kc} = \frac{1 + \sum_i \mathbf{1}\{g_i = k, r_i = c\}}{C + \sum_i \mathbf{1}\{g_i = k\}}.$$

The returned probability is the unweighted arithmetic average of the three outputs: $\hat{P}_i = \{P_{\mathrm{LDA}}(U_i) + Q_{g_i,:} + P_{\mathrm{logit}}(U_i)\}/3$. In-sample KMeans assignments are used to form Q; this procedure is an ensemble over transformed OOF predictions, not another OOF layer for the three meta-level components. Code and data constraints for individual StatCore classifiers apply.

E. Temperature-calibrated stack

The feature matrix is the ordinary OOF stack Z. To estimate a scalar temperature without scoring the in-sample fit of a logistic model, the implementation partitions Z, y into $K' = \min(K, 3)$ stratified folds, fits the same logistic meta-learner on each complement, and stores held-out decision scores $s_i \in \mathbb{R}^C$. It minimizes

$$\hat{\eta} = \arg \min_{\eta \in [-3,3]} -\frac{1}{n} \sum_i \log\left[\mathrm{softmax}(s_i / e^\eta)_{r_i}\right], \qquad \hat{T} = e^{\hat{\eta}},$$

using a 60-iteration golden-section search. The final logistic meta-estimator is refit on all (Z, y); inference applies $\mathrm{softmax}(s(x)/\hat{T})$. This adds cross-fitting at the meta-model calibration stage, but the original base-level OOF features and their global data-dependent construction remain the same. The optimized in-sample or calibration-fold loss is not a final test result; calibration must be externally assessed.

IV. Reference baseline and comparison boundary

Wolpert's stacked generalization and Breiman's stacked regression motivate learning combinations of generalizers [1], [2]. A standard `StackingClassifier` is the sklearn comparator: its default meta-classifier and splitting behavior should be reported explicitly when used, and all preprocessing must be inside the fold-fitted estimator pipeline [3]. Fair comparisons hold fixed training data, folds, base learners, probability semantics, and primary metric. StatCore's NumPy implementation is not inherently superior because it avoids a numerical dependency; sklearn-compatible wrappers still import sklearn. No claim of superiority follows from code structure alone.

TABLE I
IMPLEMENTED STATCORE META-VARIANT MECHANICS. ALL VARIANTS REFIT BASE MODELS ON THE FULL SUPPLIED TRAINING DATA FOR PREDICTION.

Variant	Meta-feature / score	Combination	Important caveat
Performance-aware	OOF log-loss per base	Temperature softmax weighted probability average	Weights use the same labels being modeled
Metrics-based	F1-scaled OOF probability, confidence, entropy, margins, consensus agreement	Logistic regression	Macro-F1 is a global statistic computed on OOF rows
OOF stacking	Concatenated class probabilities	Logistic regression	Canonical cross-fitted stack; correlated bases can add variance
Statistical meta	Concatenated clipped log probabilities	Mean of LDA, Laplace cluster posterior, logistic output	Cluster table uses training assignments
Calibrated stack	Concatenated class probabilities	Logistic regression then scalar temperature	Temperature fit uses cross-fitted meta scores; requires external calibration evaluation

V. SKLEARN-COMPATIBLE STATCORE ADAPTERS

`statcore_estimators.py` defines `StatCoreLogisticClassifier`, `StatCoreLDAClassifier`, `StatCoreKNNClassifier`, `StatCoreMLPClassifier`, and `StatCoreMetaClassifier`. They inherit the standard `BaseEstimator`/`ClassifierMixin` protocol and a shared adapter that validates arrays, records input names and class labels, builds and fits a corresponding StatCore model, normalizes clipped nonnegative probability rows, and maps argmax columns back to `classes_`. Constructor fields are direct estimator parameters to support sklearn `clone` and search. The adapter does not make the numeric core sklearn-free as a package; it is an optional integration layer. Default meta bases are StatCore logistic, LDA, and KNN adapters; StatCore MLP is separately available and is included in AutoML's pool. Feature importance mixin behavior is inherited from the broader ReliableML meta-learning module.

VI. METRIC DEFINITIONS AND EDGE CASES

The following are exact shared definitions in `evaluation/metrics.py`. For classification, the metric class set is $\mathcal{U} = \text{unique}(y \cup \hat{y})$; one-vs-rest counts define TP_c, FP_c, FN_c. Per-class precision is $TP_c/(TP_c + FP_c)$, recall is $TP_c/(TP_c+FN_c)$, and F1 is $2P_cR_c/(P_c+R_c)$; any zero denominator yields zero. Macro scores are unweighted means over $\mathcal{U}$. Accuracy is $n^{-1}\sum_i \mathbf{1}(y_i = \hat{y}_i)$.

For the multiclass MCC, let confusion matrix $C_{ab} = \#\{i : y_i = a, \hat{y}_i = b\}$, row totals t_a, column totals p_b, and $s = \text{tr}(C)$. Code computes

$$\text{MCC} = \frac{sn - \sum_a t_a p_a}{\sqrt{(n^2 - \sum_b p_b^2)(n^2 - \sum_a t_a^2)}};$$

it returns zero when denominator is nonpositive. This is a multiclass correlation coefficient (Gorodkin) [4].

If probabilities P are provided, log-loss is $-n^{-1}\sum_i \log(\max(P_{i,r_i}, 10^{-12}))$, with values capped above at 1 before log. Classes default to sorted unique true labels, or use explicit `classes`; unknown truth labels receive probability zero (therefore clipped penalty). ECE uses maximum confidence $q_i = \max_c P_{ic}$, predicted column $\arg\max_c P_{ic}$, and $B = 10$ equal-width bins with edges evenly spaced from 0 to 1. Bins are assigned by digitizing against interior edges; each row enters one bin. With I_b the bin's rows,

$$\text{ECE} = \sum_{b:|I_b|>0} \frac{|I_b|}{n} \left| \frac{1}{|I_b|}\sum_{i\in I_b} \mathbf{1}(\hat{c}_i = c_i) - \frac{1}{|I_b|}\sum_{i\in I_b} q_i \right|.$$

Here c_i on the indicator is the class index found by search in the provided class array; for intended sorted full class columns this is the usual correctness indicator. The implementation does not normalize or validate probability rows in this metrics function, and the caller is responsible for column/class alignment. ECE depends on bin count and is not a proper scoring rule; log-loss is proper under correct specification [5].

For regression with residual $e_i = y_i - \hat{y}_i$, $\text{MSE} = n^{-1}\sum e_i^2$, $\text{MAE} = n^{-1}\sum |e_i|$, and $R^2 = 1 - \sum e_i^2 / \sum (y_i - \bar{y})^2$, with $R^2 = 0$ for constant targets. Reconstruction metrics use element count q: $\text{MSE} = q^{-1}\|X - \hat{X}\|_F^2$, $\text{MAE} = q^{-1}\|X - \hat{X}\|_1$, and relative Frobenius error $\|X - \hat{X}\|_F/(\|X\|_F + 10^{-12})$. Clustering inertia is the sum of squared deviations within labels, relative to supplied centers indexed by integer label or empirical cluster means when centers are omitted; `n_clusters_found` $is the number of distinct labels$. Supplied center indexing assumes labels are valid center indices. Shape mismatches raise errors where coded; empty/degenerate edge behavior follows NumPy semantics rather than additional statistical conventions.

VII. ALGORITHM AND COMPLEXITY

A1. Generate deterministic stratified fold IDs; for each fold and base learner, clone, fit on complement, infer held-out rows, and align columns to the sorted global class vector.

A2. Compute OOF metrics; refit each base learner on all training rows.

A3. Construct variant-specific meta-features and fit the corresponding combiner or derive weights/posteriors. For calibrated stack, first cross-fit the logistic combiner over meta-features, optimize scalar log-temperature, then refit logistic on all rows.

TABLE II
SHARED METRIC FAMILIES

Task	Returned keys
Classification	accuracy, precision_macro, recall_macro, f1_macro, mcc
With class probabilities	log_loss, ece (in addition)
Regression	mse, mae, r2
Reconstruction	reconstruction_mse, reconstruction_mae, relative_frobenius
Clustering	inertia, n_clusters_found

A4. At prediction, check finite numeric features and fitted state, calculate full-data base outputs, perform the selected combination, and map argmax indices to original sorted classes.

If base-fit cost is $T_m(n,p)$, cross-fitting plus full refit costs $K\sum_m T_m((1-1/K)n,p) + \sum_m T_m(n,p)$, plus meta fit. Storage for OOF values is $O(MnC)$. The calibrated variant additionally fits K' logistic models and performs a fixed 60-step scalar search, each objective evaluating nC softmax terms. These are computational counts, not runtime guarantees.

VIII. Empirical protocol and measured results

The accompanying `experiments/meta_experiment.py` was executed with `.venv\Scripts\python.exe` on Python 3.15.0, NumPy 2.5.4, and scikit-learn 1.9.1. For each run, data were split by stratification into 75% training and 25% held-out rows (seed 42 or 43); base/meta OOF used three folds. The StatCore methods used its NumPy logistic, shrinkage LDA, and 5-neighbor KNN as base estimators; the explicit sklearn StackingClassifier baseline used standard-scaled logistic regression, LDA, and 5-neighbor KNN with a logistic final estimator. Table III reports held-out accuracy/log-loss/ECE as triplets. Values are observed on these fixed splits, not estimates across independent populations.

The Wine test split contained 45 rows and the Breast Cancer split 143 rows. The best accuracy varied by seed and dataset; the sklearn comparator sometimes had lower log-loss than StatCore variants. The temperature procedure did not uniformly improve held-out log-loss, as expected for a calibration parameter fitted on limited cross-fitted training scores. These numbers are descriptive and do not demonstrate a significant advantage. The same dataset reused across seeds does not create independent experimental units, and all choices made after observing these outputs would need an untouched evaluation.

IX. Limitations

A benchmark is specific to datasets and split protocol and does not establish population-level superiority. Model variants, datasets, and seeds considered after seeing results create further selection bias. Calibrated probabilities require independent calibration assessment; class imbalance favors reporting balanced metrics alongside accuracy; repeated seeds on one dataset are not independent datasets.

X. Conclusion

StatCore MetaLearner implements five distinct but related probability-combination procedures. The formal distinctions here follow the exact source behavior: cross-fitted base outputs, estimator-level metric transformations, a logistic stack, a three-component statistical ensemble, and cross-fitted temperature scaling. The shared metric formulas expose conventions relevant to reproducibility. OOF construction is a leakage-control measure, not a guarantee of unbiased generalization.

References

[1] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[2] L. Breiman, "Stacked regressions," *Machine Learning*, vol. 24, pp. 49–64, 1996.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] J. Gorodkin, "Comparing two K-category assignments by a K-category correlation coefficient," *Computational Biology and Chemistry*, vol. 28, no. 5–6, pp. 367–374, 2004.

[5] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[6] J. Monchwe *et al.*, ReliableML, software repository, 2026. https://github.com/Jabualani/ML-Reliabilty.

TABLE III
STATCORE META-LEARNING HELD-OUT RESULTS (ACCURACY / LOG-LOSS / ECE; SEED-SPECIFIC STRATIFIED 25% TEST SPLITS).

Method	Wine, seed 42	Wine, seed 43	Breast Cancer, seed 42	Breast Cancer, seed 43
Performance-aware	1.000 / 0.078 / 0.066	0.911 / 0.150 / 0.060	0.979 / 0.088 / 0.044	0.958 / 0.137 / 0.033
Metrics-based	0.956 / 0.208 / 0.035	0.933 / 0.224 / 0.054	0.979 / 0.067 / 0.013	0.951 / 0.112 / 0.028
OOF stacking	0.978 / 0.095 / 0.031	0.911 / 0.249 / 0.066	0.979 / 0.070 / 0.014	0.951 / 0.115 / 0.020
Statistical meta	0.933 / 0.176 / 0.069	0.956 / 0.186 / 0.043	0.986 / 0.095 / 0.043	0.944 / 0.150 / 0.023
Calibrated stack	0.978 / 0.098 / 0.020	0.911 / 0.341 / 0.081	0.979 / 0.067 / 0.008	0.951 / 0.118 / 0.031
sklearn stacking baseline	0.956 / 0.071 / 0.054	0.956 / 0.128 / 0.068	0.972 / 0.073 / 0.045	0.965 / 0.117 / 0.019